INTRODUCTION OF PROGRAMMING CONCEPTS

DAY 1

Programming Paradigm

- A **paradigm** in programming is a way of thinking about and structuring code to solve problems.
- It defines the style and approach used to write programs, such as object-oriented, procedural, or functional programming.

Classifications of Programming Paradigms

- Programming paradigms are broadly classified into two main categories:

1. **Imperative Paradigm** (How to do)
   - Focuses on step-by-step instructions for solving a problem.
   - Uses statements to change a program’s state.
   - Examples: C, Java, Python (partially), JavaScript, C++.
2. **Declarative Paradigm** (What to do)
   - Focuses on defining what the program should accomplish without specifying exact steps.
   - Example: SQL, Prolog, Haskell.

Types of **Imperative Paradigms**

1. **Procedural Programming**
   - Based on procedures (functions).
   - Code is organized into functions that perform operations on data.
   - Examples: C, Pascal.
2. **Object-Oriented Programming (OOP)**
   - Organizes code into objects that encapsulate data and behaviour.
   - Examples: Java, C++, Python, C#.
3. **Parallel Programming**
   - Focuses on running multiple computations simultaneously.
   - Examples: OpenMP, CUDA.
4. **Scripting Programming**
   - Used for automation, scripting, and quick prototyping.
   - Examples: Python, JavaScript, Bash.

Types of **Declarative Paradigms**

1. **Functional Programming (FP)**
   - Uses pure functions and avoids changing states.
   - Examples: Haskell, Lisp, Scala.
2. **Logic Programming**
   - Uses facts and rules for computation.
   - Examples: Prolog.
3. **Database Query Languages**
   - Focus on retrieving and manipulating data declaratively.
   - Examples: SQL.

Object-oriented programming (OOP)

- Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects“.

- Object may contain data, in the form of fields, often known as **attributes**; and code, in the form of procedures, often known as **methods**.

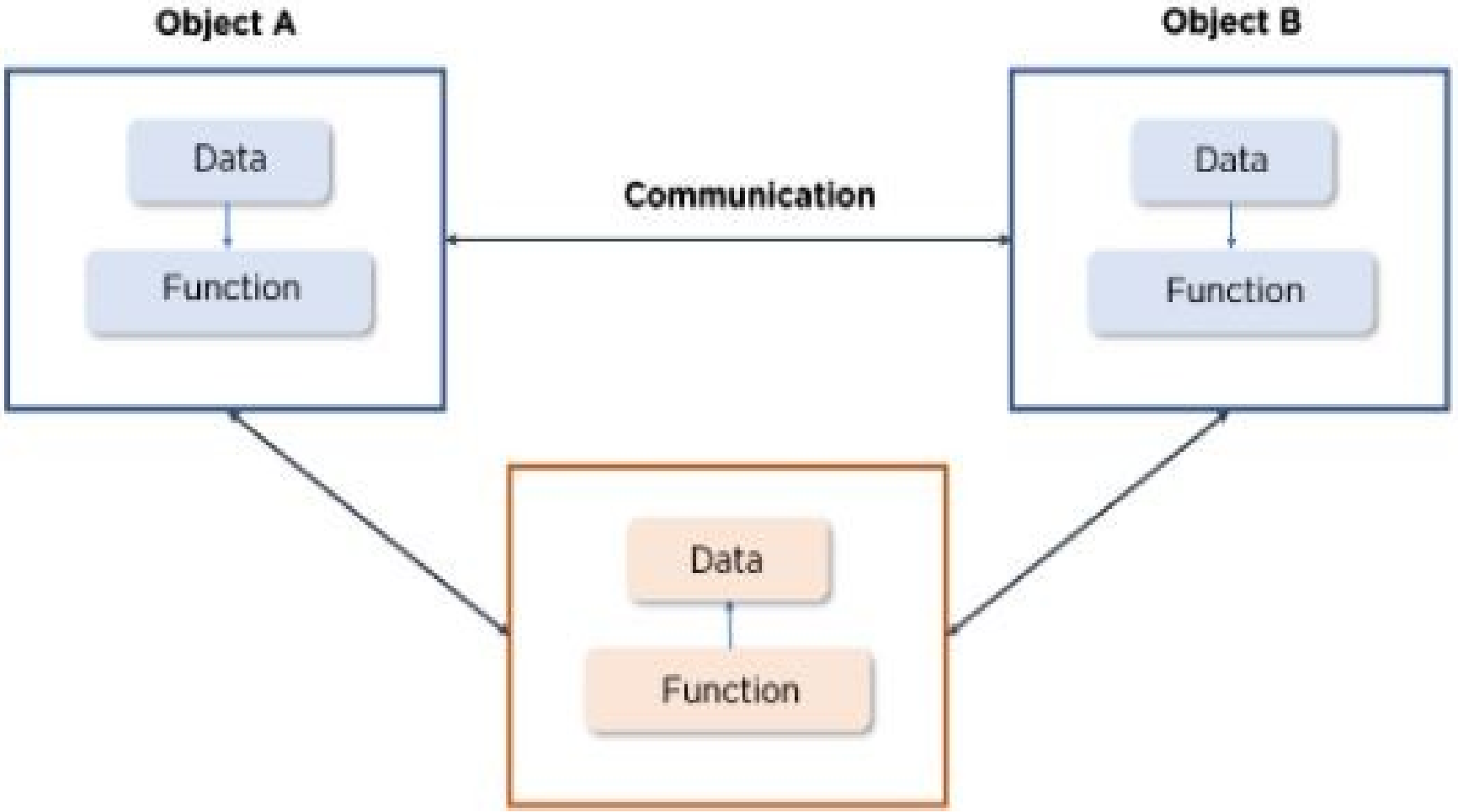
* For example, a person is an object which has certain properties such as height, gender, age, etc. It also has certain methods such as move, talk, and so on.

Object

- This is the basic unit of object-oriented programming.
- Objects are like real-life entities. They have their properties and methods.
- That is both data and function that operate on data are bundled as a unit called an. Object
- Eg:

  Consider a car as an object. The car has so many characteristics like colour, company name, modal name and price, etc.

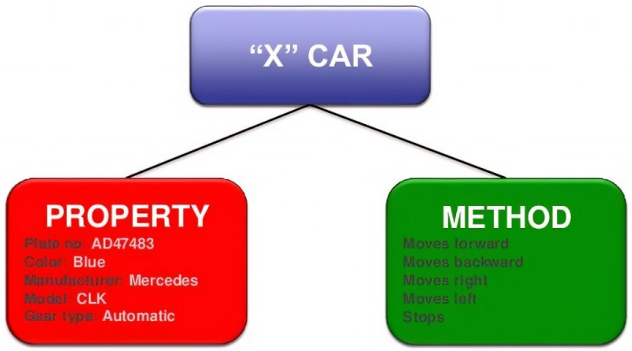
  we can perform actions like start, break, and stop these are the methods

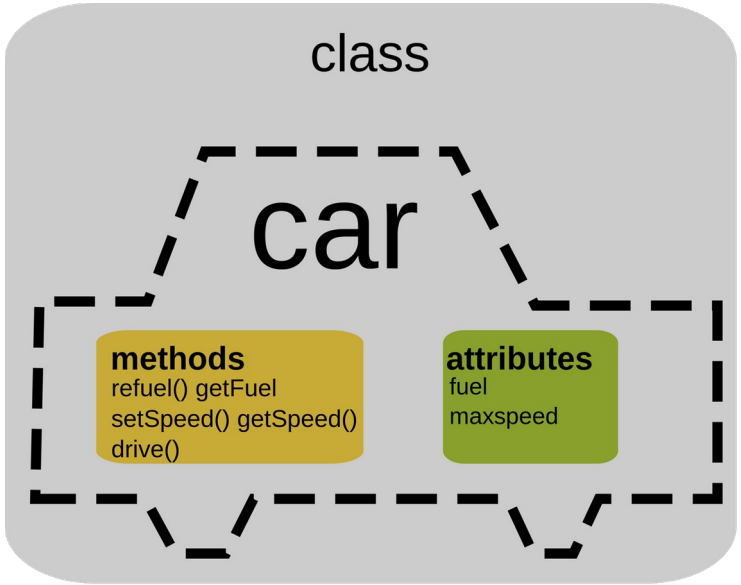

Object A
Data
Function
Communication
Object B
Data
Function
Data
Function

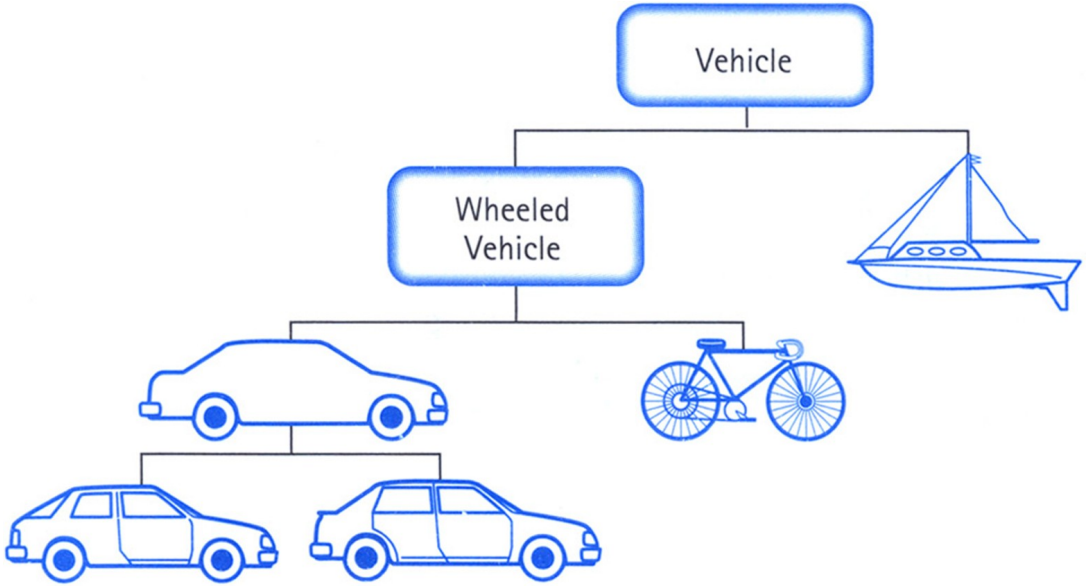
Class

- Class is a blueprint of a real-life entity.
- It describes how the object will look alike, what characteristics it holds and what kind of actions we can perform on it.

Classes & Objects







OOP has four basic concepts

- **Abstraction**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**

Abstraction

* It refers to, providing only essential information to the outside world and hiding their background details.



Encapsulation

* Encapsulation is a process of binding data members (variables, properties) and member functions (methods) into a single unit.

* It is also a way of restricting access to certain properties or component. The best example for encapsulation is



Inheritance

* The ability to create a new class from an existing class is called Inheritance.

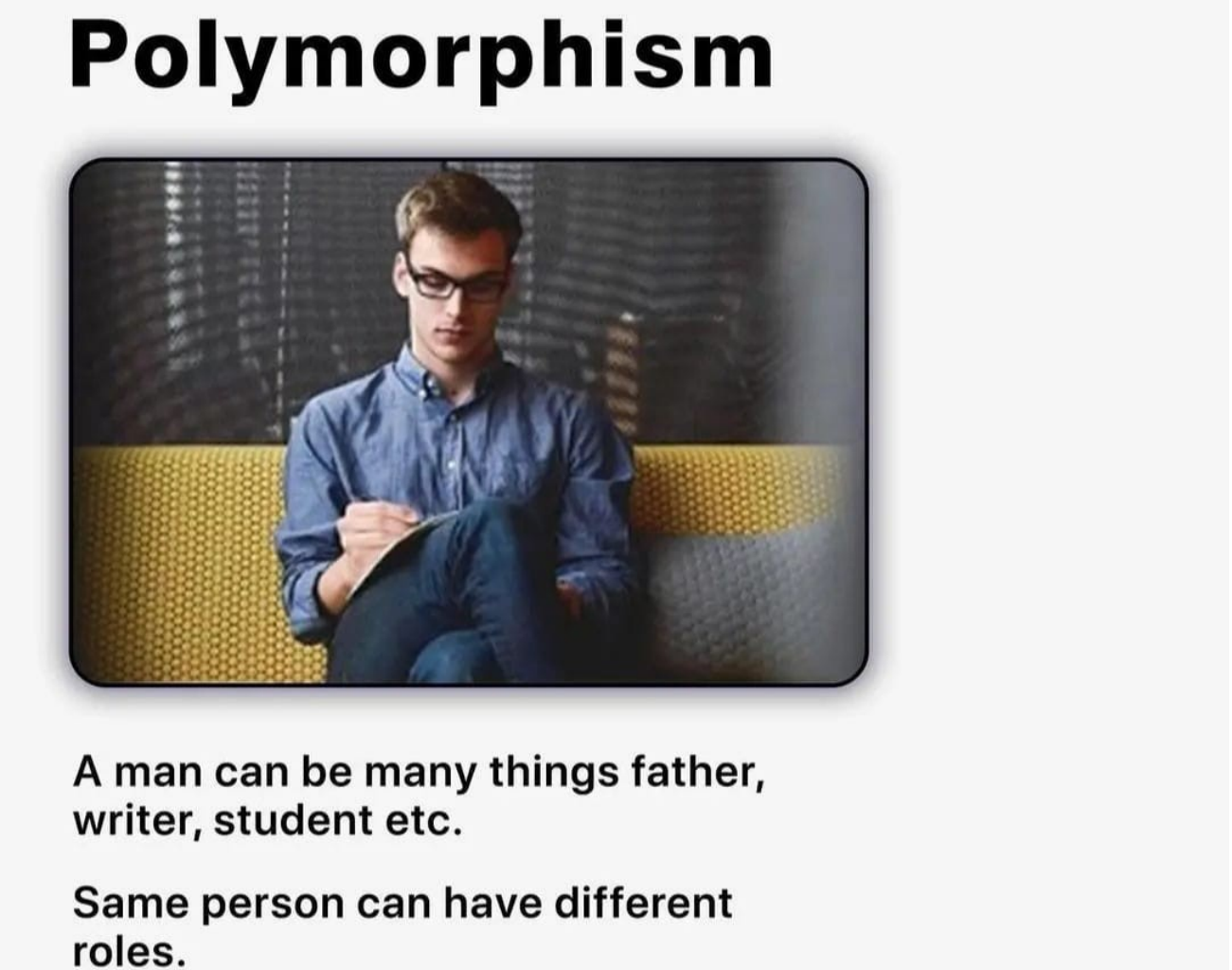
* Using inheritance, we can create a Child class from a Parent class such that it inherits the properties and methods of the parent class and can have its own additional properties and methods.



Polymorphism

* The word polymorphism means having many forms.

* Polymorphism occurs when there is a hierarchy of classes and they are related by inheritance.

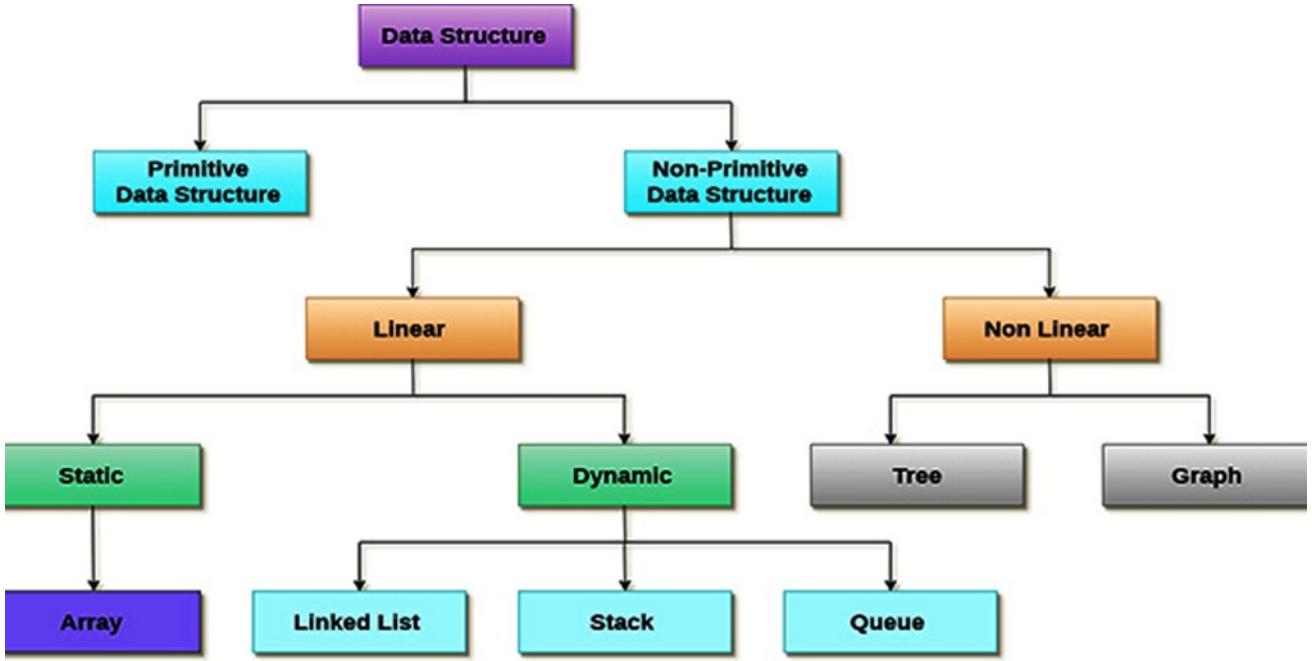


Data structure

- Data structure is a particular way of storing and organizing data in a computer so that it can be used efficiently.

- eg: Array, stack, tree, heap, hash table.

Data structure

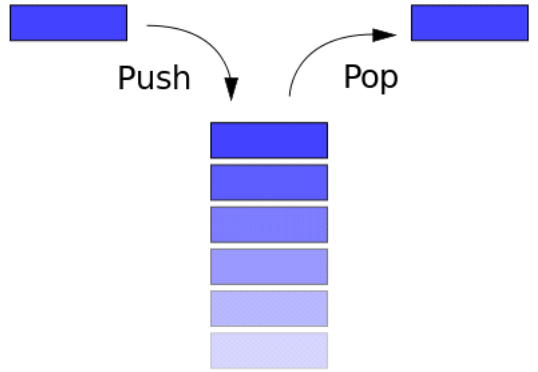


Stack

* A stack is a particular data type or collection in which the main operations are the addition of an item, known as push, and removal of it, known as pop.

* Stacks implement a **LIFO** (Last In First Out) structure which means that the last element added to the structure must be the first one to be removed.

* Memory Management: In JavaScript, the stack is used for storing simple data types like numbers, booleans, and references to objects (primitive data types). Function calls, along with their local variables, are also stored in the stack. When a function is called, a new stack frame is created, and when the function completes, the stack frame is popped off the stack.



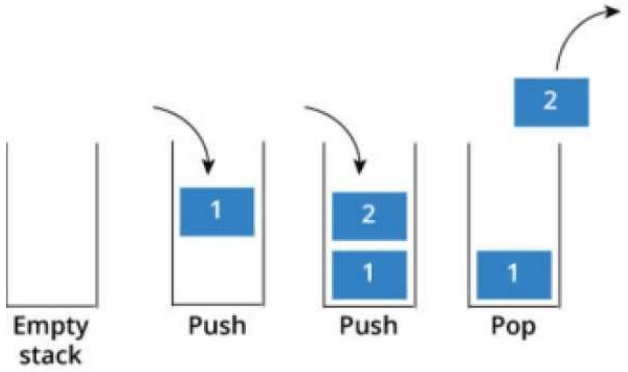
Heap

- The heap is an unstructured memory pool where data can be stored and accessed in a more flexible manner.
- **Memory Management**: In JavaScript, the heap is used to allocate memory for objects, arrays, and functions—essentially anything that requires more memory than what's available on the stack.
- The heap allows for dynamic memory allocation, meaning that the size of these structures can grow as needed.

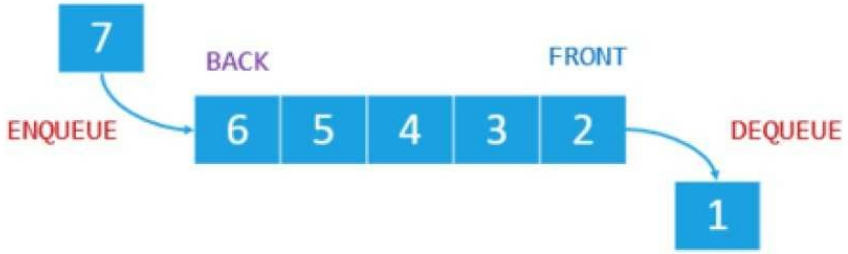
Queue

* Queues are collection that keep objects in a certain order while applying the FIFO (First in First out) format. It is just like a line of people, except that data doesn't cut in it.

* Adds an item at the front of the queue. The process is the same as PUSH from stack, but I changed for the sake of the exercise.



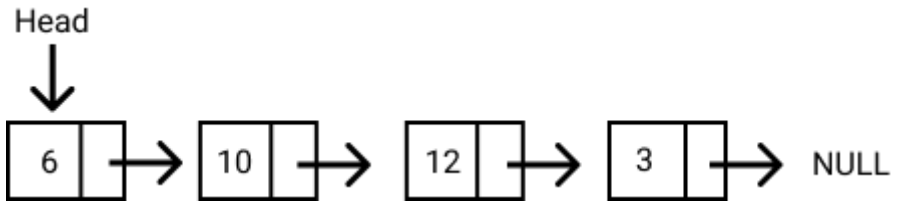
Stack



Queue

Linked list

- Linked lists are data structures that are made of groups of nodes which together represent a sequence.
- A linked list is a linear data structure where each element (called a node) contains a value and a reference (or link) to the next node in the sequence.
- JavaScript doesn’t have a built-in linked list, but it can be implemented manually.



Algorithm

- Writing a logical step-by-step method to solve the problem is called the algorithm.
- In other words, an algorithm is a procedure for solving problems.
- In order to solve a mathematical or computer problem, this is the first step in the process.
- An algorithm includes calculations, reasoning, and data processing. Algorithms can be presented by natural languages, pseudocode, and flowcharts, etc.

Flowchart

- Flowcharting is the process of drawing a flowchart for an algorithm. The flowchart uses various symbols in the representation.

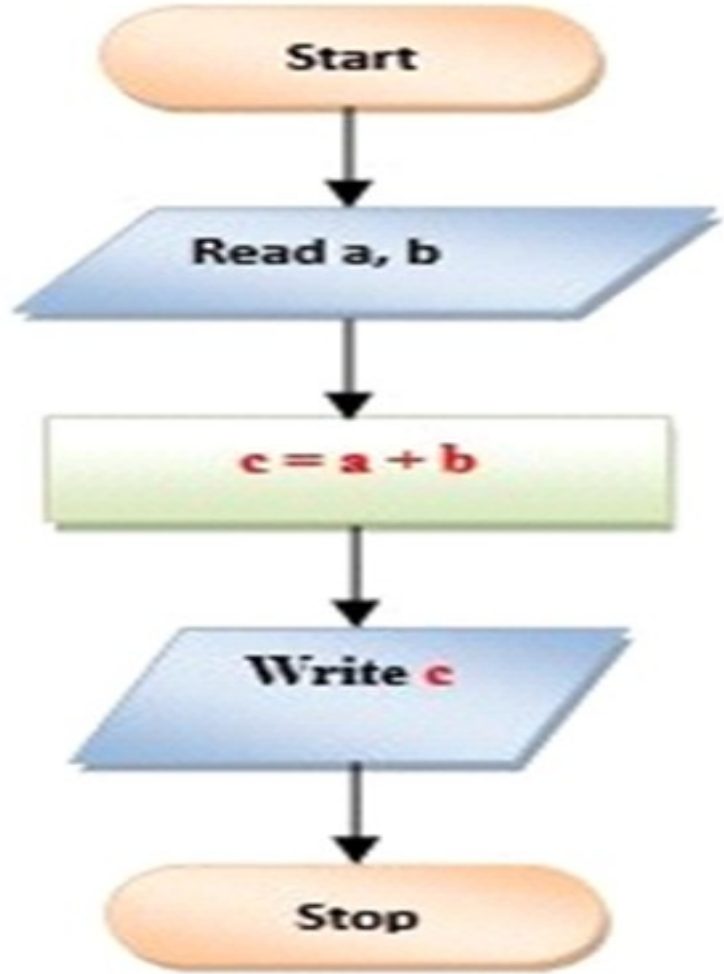


To find sum of two numbers

Algorithm

1. Start
2. Read a, b
3. c = a + b
4. Print or display c
5. Stop

Flowchart



Algorithm	Flowchart
It is a procedure for solving problems.	It is a graphic representation of a process.
The process is shown in step-by-step instruction.	The process is shown in block-by-block information diagram.
It is complex and difficult to understand.	It is intuitive and easy to understand.
It is convenient to debug errors.	It is hard to debug errors.
The solution is showcased in natural language.	The solution is showcased in pictorial format.
It is somewhat easier to solve complex problem.	It is hard to solve complex problem.
It costs more time to create an algorithm.	It costs less time to create a flowchart.

General IT & Software Terms

- **Algorithm** – A step-by-step procedure to solve a problem
- **Bug** – An error in a program that needs fixing
- **Code Editor** – A software to write and edit code (VS Code, Sublime Text)
- **IDE** (Integrated Development Environment) – A complete coding environment with tools (VS Code, IntelliJ, etc.)
- **Repository** – A storage location for code (GitHub, GitLab)
- **Version Control** – A system to track changes in code (Git, GitHub)

Networking & Security Terms

- **IP Address** – A unique address for a device on the internet
- **DNS (Domain Name System)** – Converts website names into IP addresses
- **SSL (Secure Sockets Layer) – Encrypts data for secure communication (HTTPS)**
- **Firewall** – A security system that blocks unauthorized access

Databases & Storage

- **Database** – A structured collection of data (SQL, MongoDB)
- **SQL (Structured Query Language)** – A language to interact with databases
- **NoSQL** – A flexible database type (MongoDB, Firebase)
- **Local Storage & Cookies** – Ways to store data in a browser

Code Execution Models in Programming

- When we write code in any programming language, it needs to be **translated into machine language** so the computer can understand and execute it.
- This process of translation and execution is called a **Code Execution Model**.
- There are mainly three ways this happens:

1. **Compilation**
2. **Interpretation**
3. **JIT (Just-In-Time) Compilation**

What is Compilation?

- **Compilation** is the process of **translating the entire source code** (like C, C++) into **machine code (binary) before** running the program.
- This translation is done by a software called a **compiler**.

Features of Compilation:

- Fast execution (after compilation).
- Errors are shown all at once after compiling.
- Example languages: **C, C++, Java (compiled to bytecode)**

What is Interpretation?

- **Interpretation** means **executing code line-by-line** directly without converting it entirely to machine code first.
- This is done by an **interpreter**.

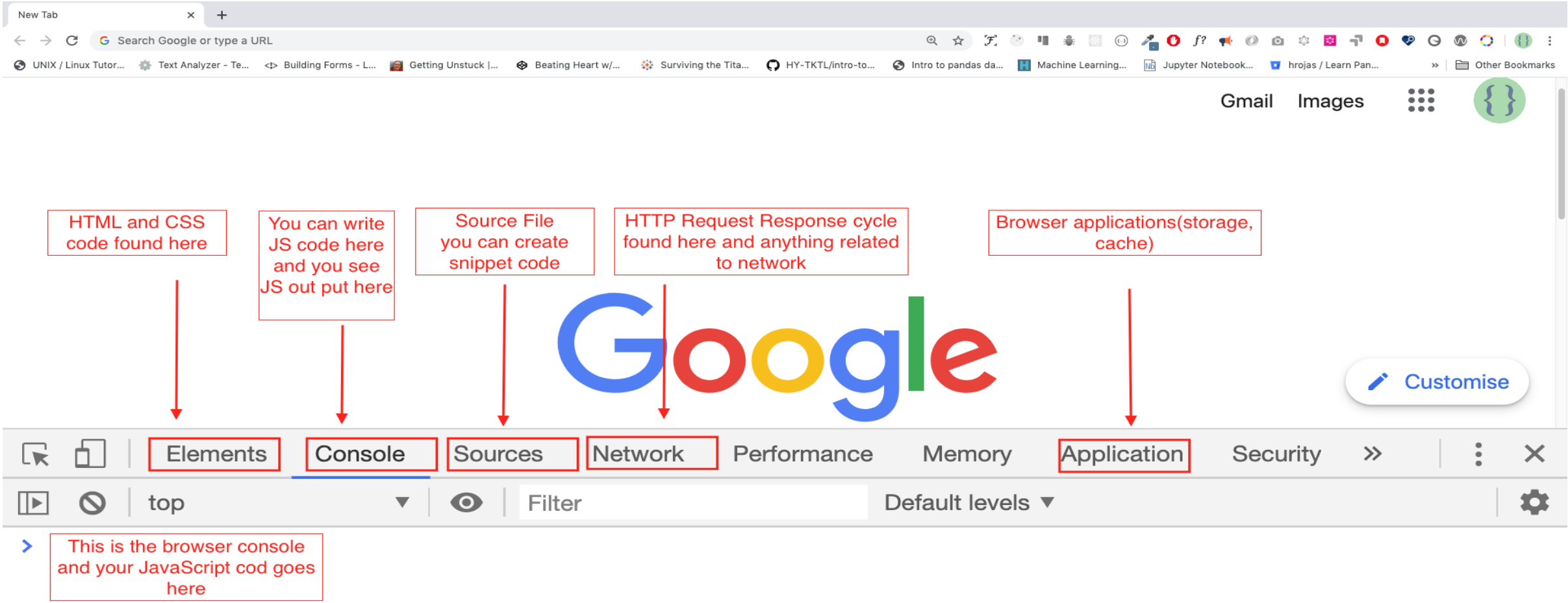
Features of Interpretation:

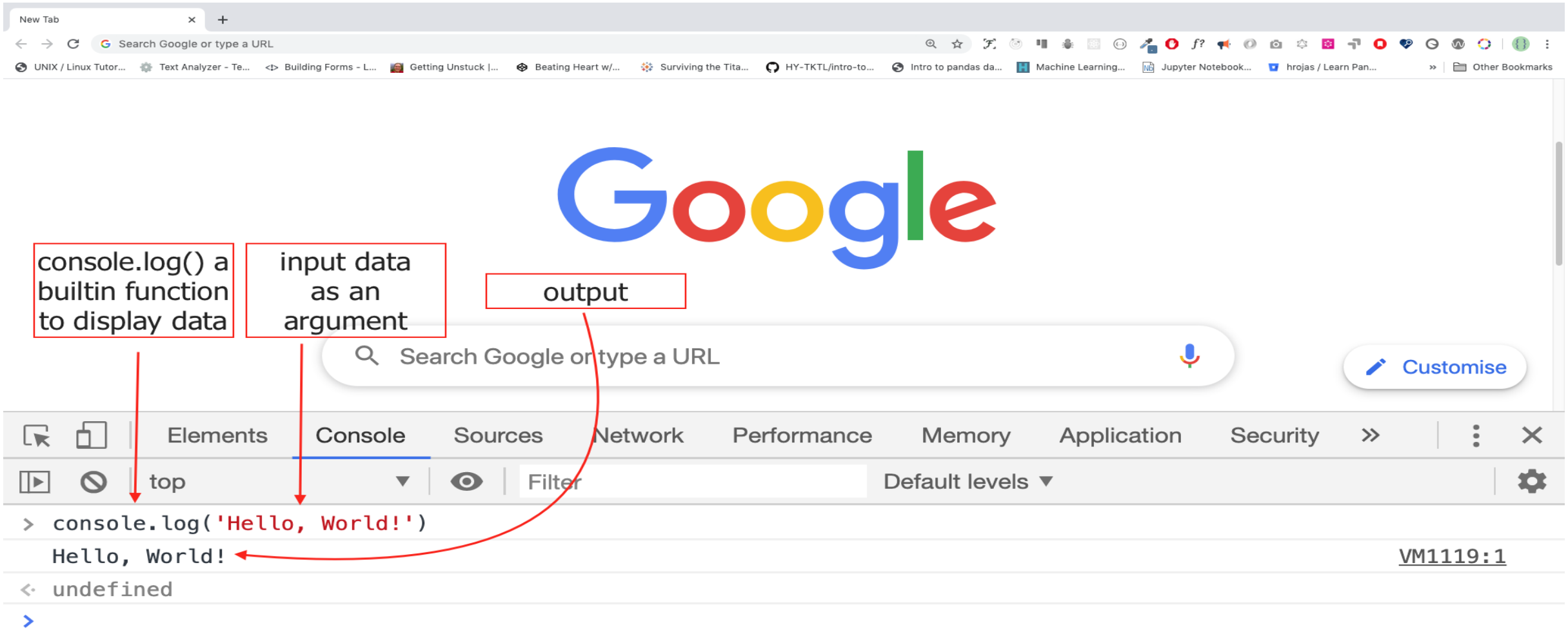
- Slower than compiled languages.
- Errors are shown **one at a time** during execution.
- Example languages: **Python, JavaScript (traditionally)**

What is JIT (Just-In-Time) Compilation?

- It’s a **combo** of both!
- Used in **modern JavaScript engines**.
- First, interprets → Then, frequently used code is **compiled during execution** (just-in-time).
- Balances speed and flexibility.

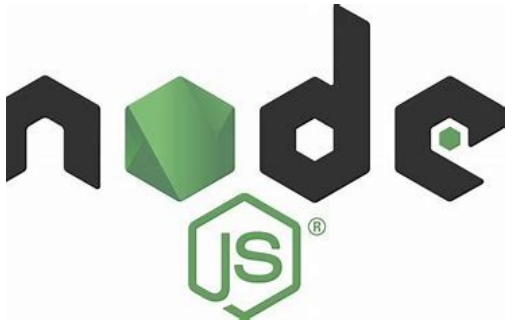
JAVASCRIPT


New Tab
Search Google or type a URL
UNIX / Linux Tutor...
Text Analyzer - Te...
Building Forms - L...
Getting Unstuck |...
Beating Heart w/...
Surviving the Tita...
HY-TKTL/intro-to...
Intro to pandas da...
Machine Learning...
Jupyter Notebook...
hrojas / Learn Pan...
Other Bookmarks
Gmail
Images
HTML and CSS code found here
You can write JS code here and you see JS out put here
Source File you can create snippet code
HTTP Request Response cycle found here and anything related to network
Browser applications(storage, cache)
Google
Customise
Elements
Console
Sources
Network
Performance
Memory
Application
Security
top
Filter
Default levels
This is the browser console and your JavaScript cod goes here


New Tab
Search Google or type a URL
UNIX / Linux Tutor...
Text Analyzer - Te...
Building Forms - L...
Getting Unstuck |...
Beating Heart w/...
Surviving the Tita...
HY-TKTL/intro-to...
Intro to pandas da...
Machine Learning...
Jupyter Notebook...
hrojas / Learn Pan...
Other Bookmarks
Google
console.log() a builtin function to display data
input data as an argument
output
Search Google or type a URL
Customise
Elements
Console
Sources
Network
Performance
Memory
Application
Security
top
Filter
Default levels
> console.log('Hello, World!')
Hello, World!
VM1119:1
undefined
>

NodeJs

- What is Node.js?
- Node.js is an open source server environment
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server
- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database



- Official Node.js website has installation instructions for Node.js: https://nodejs.org

JavaScript

- JavaScript is a **versatile programming language** used for both frontend and backend development.
- **Frontend (Client-side)**: JavaScript is used to create interactive and dynamic elements on web pages, such as dropdown menus, form validations, and content updates without reloading the page. It runs directly in the user's browser.
- **Backend (Server-side)**: JavaScript, with the help of environments like Node.js, can also be used on the server to handle database operations, manage server logic, and process requests from the client, making it a full-stack language.

JavaScript Compilation

- JavaScript has no compilation step. Instead, an interpreter in the browser reads over the JavaScript code, interprets each line, and runs it.
- More modern browsers use a technology known as **Just-In-Time (JIT)** compilation, which compiles **JavaScript to executable bytecode** just as it is about to run.
- Runs in the browser (via the **JavaScript engine**) or on the server (e.g., using **Node.js**).

History of JavaScript

1995:

- Developed by **Brendan Eich** at Netscape.
- Originally named **Mocha**, later renamed to **LiveScript**, and finally **JavaScript**.
- Introduced as a scripting language for Netscape Navigator.

1996:

- Microsoft created a JavaScript variant called **JScript** for Internet Explorer.
- The competition between Netscape and Microsoft led to JavaScript becoming more standardized.

1997:

- JavaScript was standardized as **ECMAScript 1 (ES1)** by ECMA International.
- This created a common standard for all browsers.

1999:

- **ECMAScript 3 (ES3)** was released, bringing **regular expressions**, **try-catch error handling**, and better string manipulation.

2009:

- **ECMAScript 5 (ES5)** introduced strict mode, JSON support, and array methods like map(), filter(), and reduce().

2015:

- **ECMAScript 6 (ES6)** marked a significant leap forward with features like:
  - let and const for block scoping.
  - Arrow functions.
  - Classes and modules.
  - Promises for asynchronous programming.

Current Version (ECMAScript 2024): (ES15)

- Published in **June 2024**.
- Annual ECMAScript updates introduce incremental improvements.
- Examples of modern features:
  - Async/await (2017).
  - Optional chaining (2020).
  - Object.groupBy and Map.groupBy methods.
  - Promise.withResolvers for easier promise management.

Current Status:

- JavaScript is one of the most widely used programming languages in the world.
- It powers modern web development frameworks like **React**, **Angular**, and **Vue**, and is essential for building scalable, interactive applications.

1.JavaScript Programs

* A computer program is a list of "instructions" to be "executed" by a computer.

* In a programming language, these programming instructions are called statements.

* A JavaScript program is a list of programming statements.

2. JavaScript Statements

- JavaScript statements are composed of: **Values, Operators, Expressions, Keywords, and Comments**.
- Most JavaScript programs contain many JavaScript statements.
- The statements are executed, one by one, in the same order as they are written.

3. Semicolons ;

* Semicolons separate JavaScript statements.

* Add a semicolon at the end of each executable statement:

* **End of Statements**: Generally, you use a semicolon to terminate a statement.

```
let x = 5;
let y = 10;
console.log(x + y);
```

- **Multiple Statements on One Line:** `let a = 1; let b = 2; let c = a + b;`

4. Keywords

- JavaScript statements often start with a keyword to identify the JavaScript action to be performed.
- JavaScript keywords are reserved words. Reserved words cannot be used as names for variables

const	Declares a block constant
function	Declares a function
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop

5. Variables

* Variables are containers that store values.

* JavaScript uses the keywords var, let and const to declare variables.

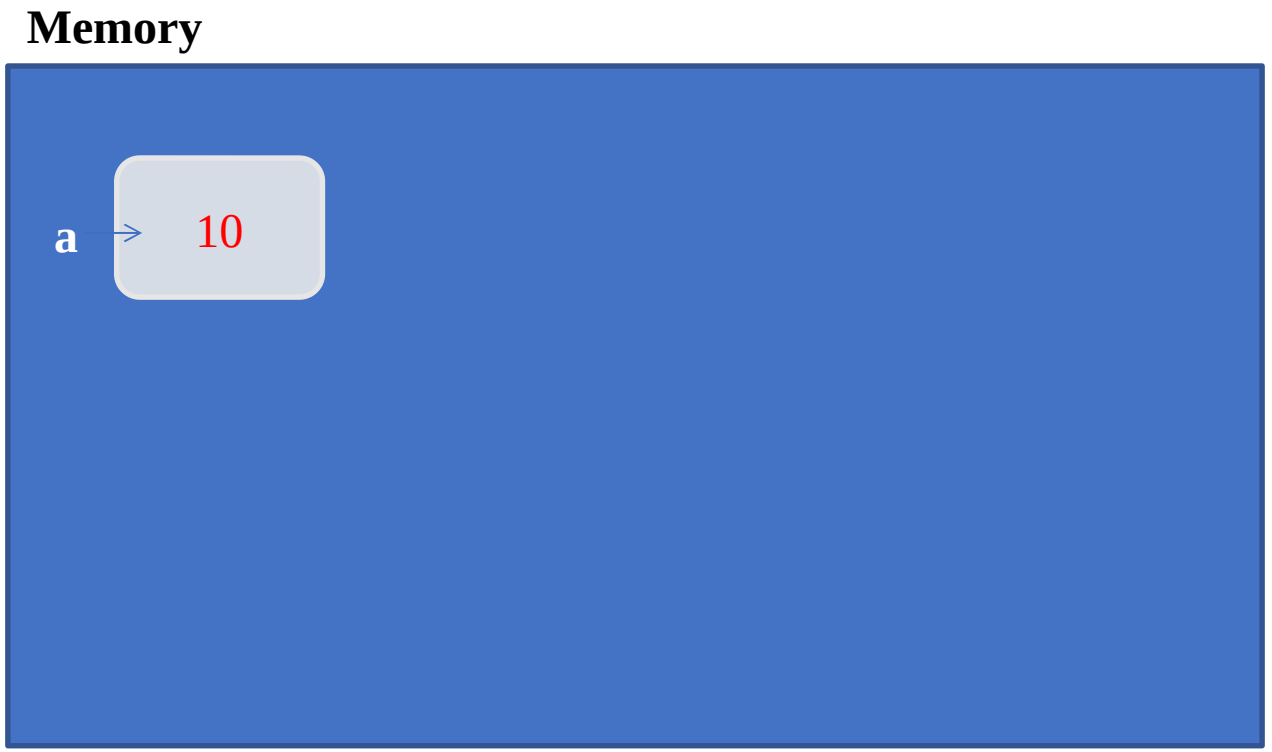
* Variables allow computers to store and manipulate data in a dynamic fashion. They do this by using a "label" to point to the data rather than using the data itself.

* By declaring a variable with the let keyword, followed by the name you give to the variable:

- let myVariable;

- It’s a named memory location
- A variable is only a name given to a memory location, all the operations done on the variable effects that memory location.
- The value stored in a variable can be changed during program execution.

var a = 10



4 Ways to Declare a JavaScript Variable:

- Using nothing
- **var:** Introduced in ES1, function-scoped or globally scoped.
- **let:** Introduced in ES6, block-scoped and allows reassignment.
- **const:** Introduced in ES6, block-scoped and cannot be reassigned.

Declaring a variable

Declaring single variable

```
var name;
```

Declaring multiple variables

```
var name, title, num;
```

Initializing a variable

* An equal sign is used to assign values to variables.

* After declaring a variable, you can give it a value:

```
myVariable = 20;
```

Variable **Hoisting** in JavaScript

- **What is Hoisting?**

  Hoisting is a JavaScript mechanism where **variable and function declarations are moved to the top** of their scope during the compilation phase, **before the code is executed**.

- Declarations are hoisted, but initializations are not. This means you can use a variable before it is declared, but it will have the value undefined (in the case of var).

```
console.log(a); // Output: undefined

var a = 10;
```

Scope

- Scope determines the accessibility or visibility of variables to JavaScript.
- There are three types of scope in JavaScript:

1. Global scope
2. Function (local) scope
3. Block scope (new with ES6)

Global scope

- Variables declared outside a function are in the global scope.
- Global variables can be accessed and changed in any other scope.

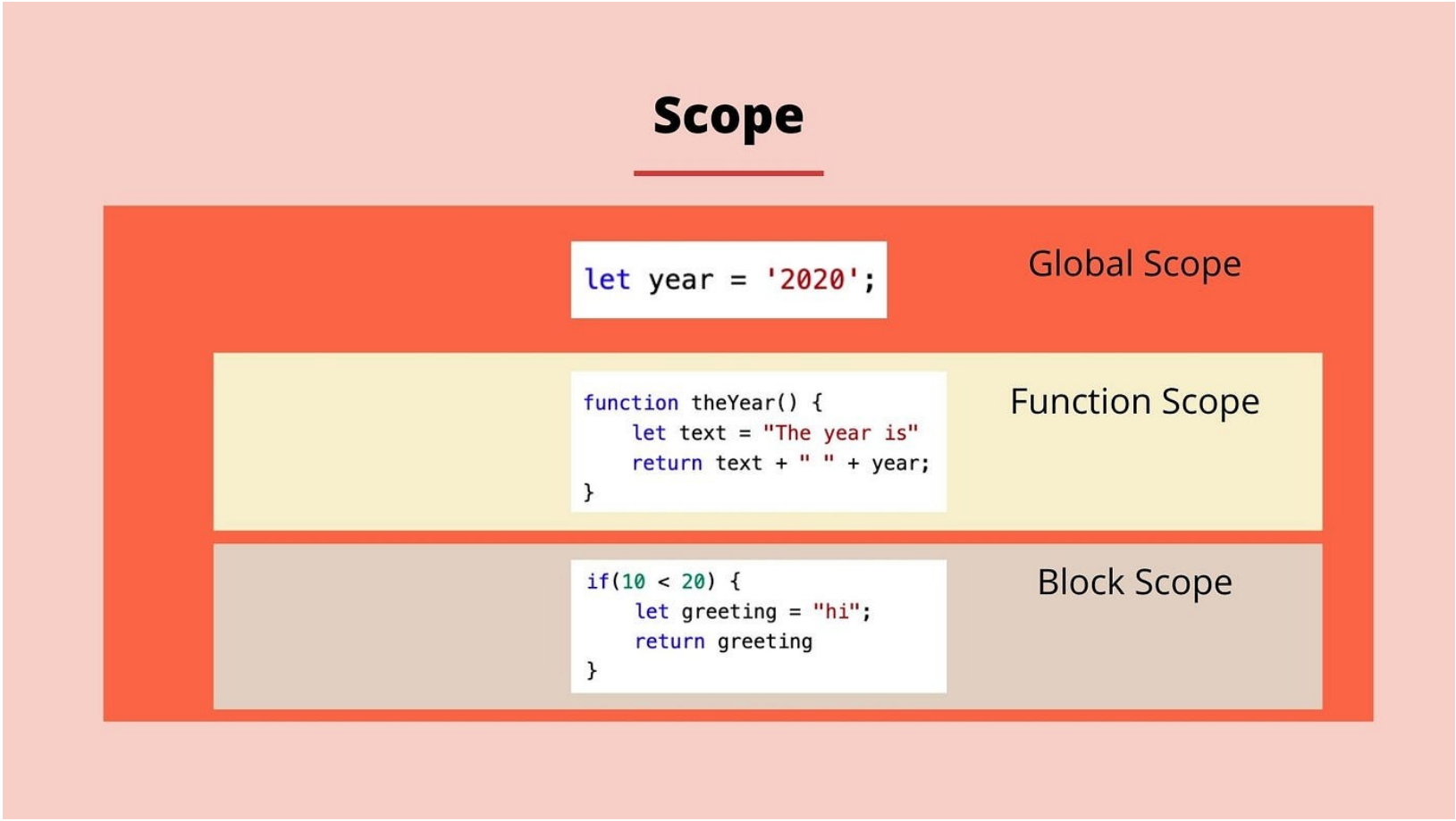
Function (local) scope

- Variables defined within a function are in local scope and are not accessible in other functions.
- Each function, when invoked, creates a new scope, therefore variables with the same name can be used in different functions.

Block scope (new with ES6)

- Block scope includes if statements and loops, or any other code wrapped in {}.
- When invoked, they don’t create a new scope.
- Variables declared inside a block scope will remain in the scope they were already in.

Example


Scope
let year = '2020';
Global Scope
function theYear() {
let text = "The year is"
return text + " " + year;
}
Function Scope
if(10 < 20) {
let greeting = "hi";
return greeting
}
Block Scope

6. Identifiers

* JavaScript identifiers are the name that we give to variables, objects, functions, arrays, classes, etc. We must use a unique name so as to identify them.

* We then use the identifier to refer to the variable, functions, etc elsewhere in the program.

* There are certain rules & restrictions that we must follow when we name an identifier.

- Eg: var **message** =”hello world”;

JavaScript Identifiers Rules

1. The identifier name must be unique within the scope.
2. The first letter of an identifier should be a upper case letter, Lower case letter, underscore, dollar sign.
3. The subsequent letters of an identifier can have upper case letter, Lower case letter, underscore, dollar sign, numeric digit
4. We cannot use reserved keywords. You can find the list of keywords for here.
5. They are **case-sensitive**. For Example, sayHello is different from SayHello
6. Cannot use spaces in a identifier name.
7. Avoid using the JavaScript Reserved Keywords as identifier name

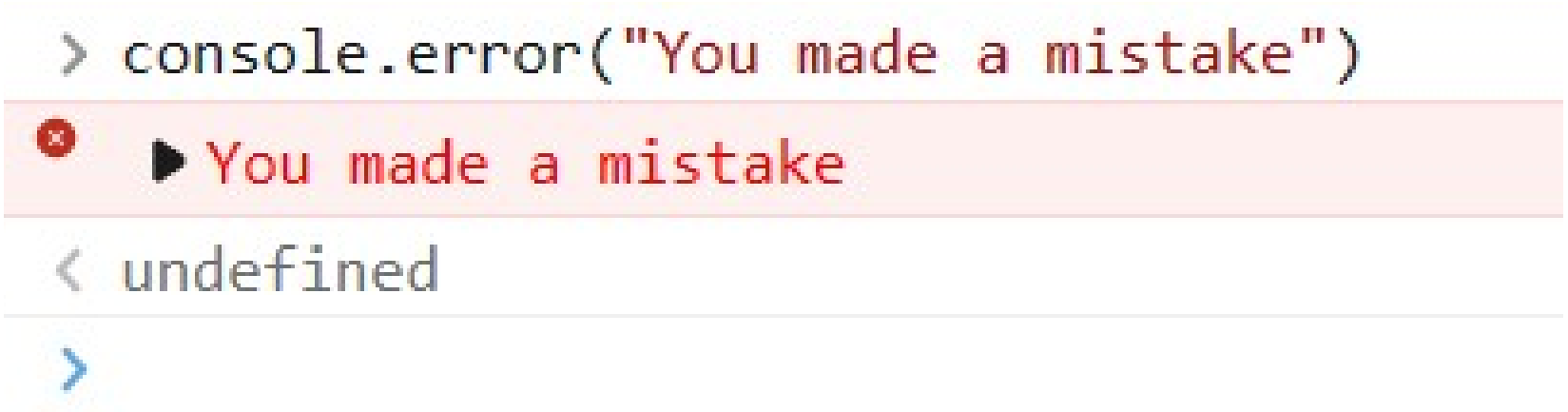
JavaScript console.log() Method

- The console.log() is a function in JavaScript that is used to print any kind of variables defined before in it or to just print any message that needs to be displayed to the user.
- Syntax:

```
console.log(“Hello World");
var a = 2;
console.log(a);
var str = “LUMINAR TECHNOLAB";
console.log(str);
```

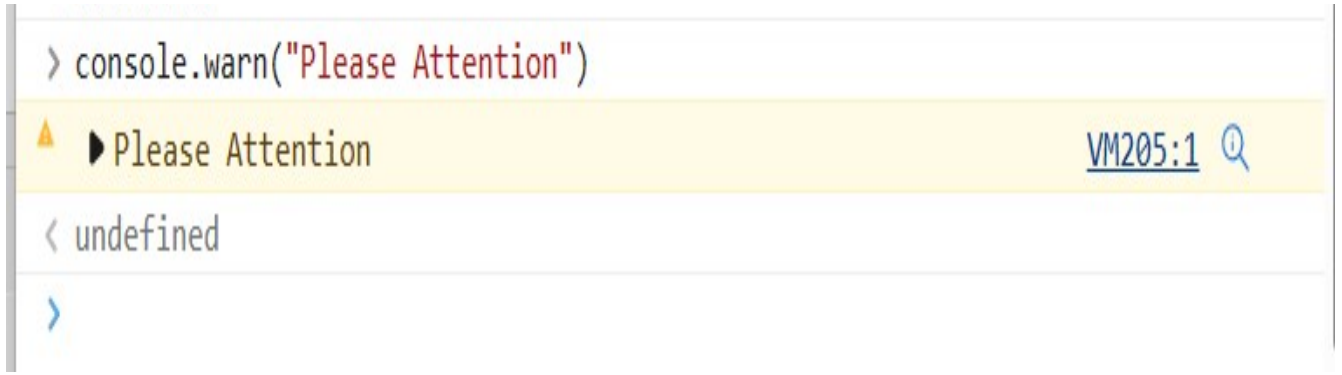
JavaScript console.error() Method

- Outputs an error message to the console



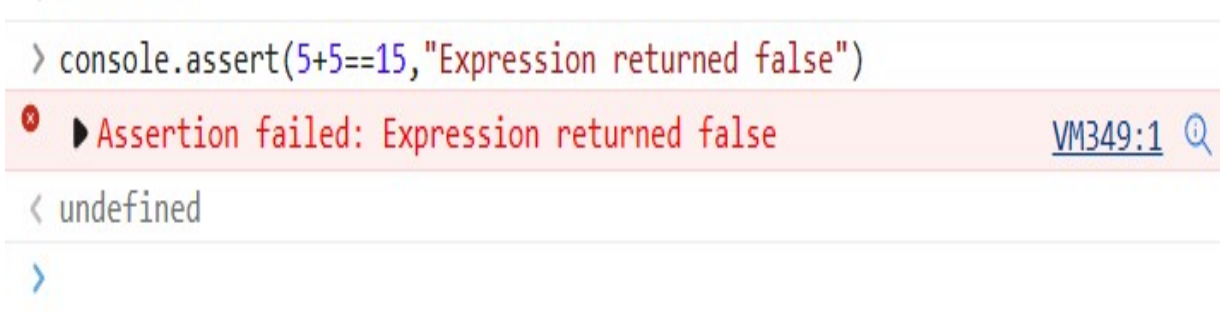
JavaScript console.warn() Method

- **Outputs a warning message to the console**



JavaScript console.assert() Method

- Writes an error message to the console if a assertion is false



JavaScript console.table() Method

```
> console.table(["Amal","Vimal"])
```

(index)	Value
0	'Amal'
1	'Vimal'

```
▶ Array(2)
< undefined
>
```

JavaScript console.clear() Method

- Clear the console – console.clear()

Comments

- Not all JavaScript statements are "executed".
- Code after double slashes // or between /* and */ is treated as a comment.
- Comments are ignored, and will not be executed

JavaScript and Camel Case

- Historically, programmers have used different ways of joining multiple words into one variable name:
- **Hyphens:**
- first-name, last-name, master-card, inter-city.
- Hyphens are not allowed in JavaScript. They are reserved for subtractions.

- **Underscore: snakecase- get_input**
- first_name, last_name, master_card, inter_city.
- **Upper Camel Case (Pascal Case):**
- FirstName, LastName, MasterCard, InterCity.
- **Lower Camel Case:**
- JavaScript programmers tend to use camel case that starts with a lowercase letter:
- firstName, lastName, masterCard, interCity.

TASKS

1. What are the features of javascript?
2. What are the main Advantages of js?
3. What JS is used for?
4. Difference between var, let, const?
5. What are most popular JS Libraries
6. What happens if you declare a variable without using var, let, or const?
7. What are reserved keywords in JavaScript? Can you give examples?
8. What is the global object in JavaScript?
9. How do block scope and function scope differ in variable declaration?